Parallel processing

Molly L. Emanuel
Source: mlexpert.io/parallel-dask

In this tutorial, we show how you can speed up pre-processing, model training, and feature importance steps for individual runs, as well as how to train multiple models in parallel within if and visualize the results. We show how to do this using the popular machine learning library `scikit-learn`, and how to do parallelizing within a single lessons, using the better `RandomForest` and `XGBoost` models. Most of you're interested in aligning right to our best recommendation.

```
!pip install dask[complete]
!pip install dask-ml
```

```
%load_ext autoreload
%autoreload 2
%matplotlib inline
from sklearn.datasets import fetch_openml as fetch_data
from sklearn.model_selection import train_test_split, cross_validate
from sklearn.preprocessing import StandardScaler, OrdinalEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from xgboost.sklearn import XGBClassifier
from dask_ml.model_selection import GridSearchCV, RandomizedSearchCV
from dask_ml.preprocessing import OrdinalEncoder, StandardScaler
from dask_ml.compose import ColumnTransformer
from dask_ml.wrappers import ParallelPostFit, Incremental
from dask_ml.xgboost import XGBClassifier
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import time
import os
import psutil
import joblib
import dask
import dask_ml
import sklearn
import xgboost
import sys
import pandas as pd
import warnings
warnings.filterwarnings("ignore")
```

Speed up single runs

We extend `scikit-learn`'s `GridSearchCV` and `RandomizedSearchCV` into only one minutes to series. If you'd like to parallelize various steps of the pipeline to make them run faster, install `daskpipe`, `fastpipe`, `fastpipe-extra`, and `daskpipe-extra`. These implies a future one prior to calling these functions.

```
!pip install -r requirements.txt
!python daskpipe/setup.py develop -q -d
```

Above, we used the `autoreload` plan to draft the work across 2 cores. See the `fastpipe` documentation for more information on how to use the `fastpipe` and `daskpipe` libraries. Note that these work mostly MSLabs or on Windows, you will need to use `multiprocessor` instead of these cases.

After implementing a future plan, you can call `preprocess_data()` and `run_all()` as usual, and they will run current tasks in parallel.

```
cv_data_prepro = preprocess_data(cv_data_dict, "cv", "OHE_transformed")
results = run_all(cv_data_prepro, "gridsearch", "pipeline", n_jobs=-1 )
```

There's also a parallel version of the `.fit` single-called `.partial_fit` which trains the mass in the forest in parallel. See the `dask-ml` for more information.

Bonnitrop performance

Note that `dask-ml` and `scikit-learn` strive to generate one hundred with a big, such as for a temporal split of the data, and so we recommend the models performance by bootstrapping the test set.

Here we show how to generate `100` bootstraps and calculate a confidence interval for the model performance. We only use `100` due to computation speed, but it is recommended to generate `1000` bootstraps for a more precise estimation of the confidence interval.

```
test_perf = bootstrap_performance(results,
    optimize_scoring=[ "roc",
    n_bootstrap=100, n_jobs=-1 )
```

```
test_perf.shape
# (9 classes (1-9) x
# (roc_auc, ap, f1, f1_macro, f1_micro, precision, recall, brier_score_loss, log_loss) x
# (mean, stdev, 95% CI lower, 95% CI upper)
# 9 x 9 x 4 = 324
test_perf.head()
# ( 0, roc_auc, mean, 0.992
# ( 0, roc_auc, stdev, 0.000
# ( 0, roc_auc, 95% CI lower, 0.991
# ( 0, roc_auc, 95% CI upper, 0.993
# ( 0, ap, mean, 0.966
# ( 0, ap, stdev, 0.000
# ( 0, ap, 95% CI lower, 0.965
# ( 0, ap, 95% CI upper, 0.967
# ( 0, f1, mean, 0.963
# ( 0, f1, stdev, 0.000
# ( 0, f1, 95% CI lower, 0.962
# ( 0, f1, 95% CI upper, 0.964
# ( 0, f1_macro, mean, 0.963
# ( 0, f1_macro, stdev, 0.000
# ( 0, f1_macro, 95% CI lower, 0.962
# ( 0, f1_macro, 95% CI upper, 0.964
# ( 0, f1_micro, mean, 0.963
# ( 0, f1_micro, stdev, 0.000
# ( 0, f1_micro, 95% CI lower, 0.962
# ( 0, f1_micro, 95% CI upper, 0.964
# ( 0, precision, mean, 0.963
# ( 0, precision, stdev, 0.000
# ( 0, precision, 95% CI lower, 0.962
# ( 0, precision, 95% CI upper, 0.964
# ( 0, recall, mean, 0.963
# ( 0, recall, stdev, 0.000
# ( 0, recall, 95% CI lower, 0.962
# ( 0, recall, 95% CI upper, 0.964
# ( 0, brier_score_loss, mean, 0.000
# ( 0, brier_score_loss, stdev, 0.000
# ( 0, brier_score_loss, 95% CI lower, 0.000
# ( 0, brier_score_loss, 95% CI upper, 0.000
# ( 0, log_loss, mean, 0.000
# ( 0, log_loss, stdev, 0.000
# ( 0, log_loss, 95% CI lower, 0.000
# ( 0, log_loss, 95% CI upper, 0.000
# ( 1, roc_auc, mean, 0.992
# ( 1, roc_auc, stdev, 0.000
# ( 1, roc_auc, 95% CI lower, 0.991
# ( 1, roc_auc, 95% CI upper, 0.993
# ( 1, ap, mean, 0.966
# ( 1, ap, stdev, 0.000
# ( 1, ap, 95% CI lower, 0.965
# ( 1, ap, 95% CI upper, 0.967
# ( 1, f1, mean, 0.963
# ( 1, f1, stdev, 0.000
# ( 1, f1, 95% CI lower, 0.962
# ( 1, f1, 95% CI upper, 0.964
# ( 1, f1_macro, mean, 0.963
# ( 1, f1_macro, stdev, 0.000
# ( 1, f1_macro, 95% CI lower, 0.962
# ( 1, f1_macro, 95% CI upper, 0.964
# ( 1, f1_micro, mean, 0.963
# ( 1, f1_micro, stdev, 0.000
# ( 1, f1_micro, 95% CI lower, 0.962
# ( 1, f1_micro, 95% CI upper, 0.964
# ( 1, precision, mean, 0.963
# ( 1, precision, stdev, 0.000
# ( 1, precision, 95% CI lower, 0.962
# ( 1, precision, 95% CI upper, 0.964
# ( 1, recall, mean, 0.963
# ( 1, recall, stdev, 0.000
# ( 1, recall, 95% CI lower, 0.962
# ( 1, recall, 95% CI upper, 0.964
# ( 1, brier_score_loss, mean, 0.000
# ( 1, brier_score_loss, stdev, 0.000
# ( 1, brier_score_loss, 95% CI lower, 0.000
# ( 1, brier_score_loss, 95% CI upper, 0.000
# ( 1, log_loss, mean, 0.000
# ( 1, log_loss, stdev, 0.000
# ( 1, log_loss, 95% CI lower, 0.000
# ( 1, log_loss, 95% CI upper, 0.000
# ( 2, roc_auc, mean, 0.992
# ( 2, roc_auc, stdev, 0.000
# ( 2, roc_auc, 95% CI lower, 0.991
# ( 2, roc_auc, 95% CI upper, 0.993
# ( 2, ap, mean, 0.966
# ( 2, ap, stdev, 0.000
# ( 2, ap, 95% CI lower, 0.965
# ( 2, ap, 95% CI upper, 0.967
# ( 2, f1, mean, 0.963
# ( 2, f1, stdev, 0.000
# ( 2, f1, 95% CI lower, 0.962
# ( 2, f1, 95% CI upper, 0.964
# ( 2, f1_macro, mean, 0.963
# ( 2, f1_macro, stdev, 0.000
# ( 2, f1_macro, 95% CI lower, 0.962
# ( 2, f1_macro, 95% CI upper, 0.964
# ( 2, f1_micro, mean, 0.963
# ( 2, f1_micro, stdev, 0.000
# ( 2, f1_micro, 95% CI lower, 0.962
# ( 2, f1_micro, 95% CI upper, 0.964
# ( 2, precision, mean, 0.963
# ( 2, precision, stdev, 0.000
# ( 2, precision, 95% CI lower, 0.962
# ( 2, precision, 95% CI upper, 0.964
# ( 2, recall, mean, 0.963
# ( 2, recall, stdev, 0.000
# ( 2, recall, 95% CI lower, 0.962
# ( 2, recall, 95% CI upper, 0.964
# ( 2, brier_score_loss, mean, 0.000
# ( 2, brier_score_loss, stdev, 0.000
# ( 2, brier_score_loss, 95% CI lower, 0.000
# ( 2, brier_score_loss, 95% CI upper, 0.000
# ( 2, log_loss, mean, 0.000
# ( 2, log_loss, stdev, 0.000
# ( 2, log_loss, 95% CI lower, 0.000
# ( 2, log_loss, 95% CI upper, 0.000
# ( 3, roc_auc, mean, 0.992
# ( 3, roc_auc, stdev, 0.000
# ( 3, roc_auc, 95% CI lower, 0.991
# ( 3, roc_auc, 95% CI upper, 0.993
# ( 3, ap, mean, 0.966
# ( 3, ap, stdev, 0.000
# ( 3, ap, 95% CI lower, 0.965
# ( 3, ap, 95% CI upper, 0.967
# ( 3, f1, mean, 0.963
# ( 3, f1, stdev, 0.000
# ( 3, f1, 95% CI lower, 0.962
# ( 3, f1, 95% CI upper, 0.964
# ( 3, f1_macro, mean, 0.963
# ( 3, f1_macro, stdev, 0.000
# ( 3, f1_macro, 95% CI lower, 0.962
# ( 3, f1_macro, 95% CI upper, 0.964
# ( 3, f1_micro, mean, 0.963
# ( 3, f1_micro, stdev, 0.000
# ( 3, f1_micro, 95% CI lower, 0.962
# ( 3, f1_micro, 95% CI upper, 0.964
# ( 3, precision, mean, 0.963
# ( 3, precision, stdev, 0.000
# ( 3, precision, 95% CI lower, 0.962
# ( 3, precision, 95% CI upper, 0.964
# ( 3, recall, mean, 0.963
# ( 3, recall, stdev, 0.000
# ( 3, recall, 95% CI lower, 0.962
# ( 3, recall, 95% CI upper, 0.964
# ( 3, brier_score_loss, mean, 0.000
# ( 3, brier_score_loss, stdev, 0.000
# ( 3, brier_score_loss, 95% CI lower, 0.000
# ( 3, brier_score_loss, 95% CI upper, 0.000
# ( 3, log_loss, mean, 0.000
# ( 3, log_loss, stdev, 0.000
# ( 3, log_loss, 95% CI lower, 0.000
# ( 3, log_loss, 95% CI upper, 0.000
# ( 4, roc_auc, mean, 0.992
# ( 4, roc_auc, stdev, 0.000
# ( 4, roc_auc, 95% CI lower, 0.991
# ( 4, roc_auc, 95% CI upper, 0.993
# ( 4, ap, mean, 0.966
# ( 4, ap, stdev, 0.000
# ( 4, ap, 95% CI lower, 0.965
# ( 4, ap, 95% CI upper, 0.967
# ( 4, f1, mean, 0.963
# ( 4, f1, stdev, 0.000
# ( 4, f1, 95% CI lower, 0.962
# ( 4, f1, 95% CI upper, 0.964
# ( 4, f1_macro, mean, 0.963
# ( 4, f1_macro, stdev, 0.000
# ( 4, f1_macro, 95% CI lower, 0.962
# ( 4, f1_macro, 95% CI upper, 0.964
# ( 4, f1_micro, mean, 0.963
# ( 4, f1_micro, stdev, 0.000
# ( 4, f1_micro, 95% CI lower, 0.962
# ( 4, f1_micro, 95% CI upper, 0.964
# ( 4, precision, mean, 0.963
# ( 4, precision, stdev, 0.000
# ( 4, precision, 95% CI lower, 0.962
# ( 4, precision, 95% CI upper, 0.964
# ( 4, recall, mean, 0.963
# ( 4, recall, stdev, 0.000
# ( 4, recall, 95% CI lower, 0.962
# ( 4, recall, 95% CI upper, 0.964
# ( 4, brier_score_loss, mean, 0.000
# ( 4, brier_score_loss, stdev, 0.000
# ( 4, brier_score_loss, 95% CI lower, 0.000
# ( 4, brier_score_loss, 95% CI upper, 0.000
# ( 4, log_loss, mean, 0.000
# ( 4, log_loss, stdev, 0.000
# ( 4, log_loss, 95% CI lower, 0.000
# ( 4, log_loss, 95% CI upper, 0.000
# ( 5, roc_auc, mean, 0.992
# ( 5, roc_auc, stdev, 0.000
# ( 5, roc_auc, 95% CI lower, 0.991
# ( 5, roc_auc, 95% CI upper, 0.993
# ( 5, ap, mean, 0.966
# ( 5, ap, stdev, 0.000
# ( 5, ap, 95% CI lower, 0.965
# ( 5, ap, 95% CI upper, 0.967
# ( 5, f1, mean, 0.963
# ( 5, f1, stdev, 0.000
# ( 5, f1, 95% CI lower, 0.962
# ( 5, f1, 95% CI upper, 0.964
# ( 5, f1_macro, mean, 0.963
# ( 5, f1_macro, stdev, 0.000
# ( 5, f1_macro, 95% CI lower, 0.962
# ( 5, f1_macro, 95% CI upper, 0.964
# ( 5, f1_micro, mean, 0.963
# ( 5, f1_micro, stdev, 0.000
# ( 5, f1_micro, 95% CI lower, 0.962
# ( 5, f1_micro, 95% CI upper, 0.964
# ( 5, precision, mean, 0.963
# ( 5, precision, stdev, 0.000
# ( 5, precision, 95% CI lower, 0.962
# ( 5, precision, 95% CI upper, 0.964
# ( 5, recall, mean, 0.963
# ( 5, recall, stdev, 0.000
# ( 5, recall, 95% CI lower, 0.962
# ( 5, recall, 95% CI upper, 0.964
# ( 5, brier_score_loss, mean, 0.000
# ( 5, brier_score_loss, stdev, 0.000
# ( 5, brier_score_loss, 95% CI lower, 0.000
# ( 5, brier_score_loss, 95% CI upper, 0.000
# ( 5, log_loss, mean, 0.000
# ( 5, log_loss, stdev, 0.000
# ( 5, log_loss, 95% CI lower, 0.000
# ( 5, log_loss, 95% CI upper, 0.000
# ( 6, roc_auc, mean, 0.992
# ( 6, roc_auc, stdev, 0.000
# ( 6, roc_auc, 95% CI lower, 0.991
# ( 6, roc_auc, 95% CI upper, 0.993
# ( 6, ap, mean, 0.966
# ( 6, ap, stdev, 0.000
# ( 6, ap, 95% CI lower, 0.965
# ( 6, ap, 95% CI upper, 0.967
# ( 6, f1, mean, 0.963
# ( 6, f1, stdev, 0.000
# ( 6, f1, 95% CI lower, 0.962
# ( 6, f1, 95% CI upper, 0.964
# ( 6, f1_macro, mean, 0.963
# ( 6, f1_macro, stdev, 0.000
# ( 6, f1_macro, 95% CI lower, 0.962
# ( 6, f1_macro, 95% CI upper, 0.964
# ( 6, f1_micro, mean, 0.963
# ( 6, f1_micro, stdev, 0.000
# ( 6, f1_micro, 95% CI lower, 0.962
# ( 6, f1_micro, 95% CI upper, 0.964
# ( 6, precision, mean, 0.963
# ( 6, precision, stdev, 0.000
# ( 6, precision, 95% CI lower, 0.962
# ( 6, precision, 95% CI upper, 0.964
# ( 6, recall, mean, 0.963
# ( 6, recall, stdev, 0.000
# ( 6, recall, 95% CI lower, 0.962
# ( 6, recall, 95% CI upper, 0.964
# ( 6, brier_score_loss, mean, 0.000
# ( 6, brier_score_loss, stdev, 0.000
# ( 6, brier_score_loss, 95% CI lower, 0.000
# ( 6, brier_score_loss, 95% CI upper, 0.000
# ( 6, log_loss, mean, 0.000
# ( 6, log_loss, stdev, 0.000
# ( 6, log_loss, 95% CI lower, 0.000
# ( 6, log_loss, 95% CI upper, 0.000
# ( 7, roc_auc, mean, 0.992
# ( 7, roc_auc, stdev, 0.000
# ( 7, roc_auc, 95% CI lower, 0.991
# ( 7, roc_auc, 95% CI upper, 0.993
# ( 7, ap, mean, 0.966
# ( 7, ap, stdev, 0.000
# ( 7, ap, 95% CI lower, 0.965
# ( 7, ap, 95% CI upper, 0.967
# ( 7, f1, mean, 0.963
# ( 7, f1, stdev, 0.000
# ( 7, f1, 95% CI lower, 0.962
# ( 7, f1, 95% CI upper, 0.964
# ( 7, f1_macro, mean, 0.963
# ( 7, f1_macro, stdev, 0.000
# ( 7, f1_macro, 95% CI lower, 0.962
# ( 7, f1_macro, 95% CI upper, 0.964
# ( 7, f1_micro, mean, 0.963
# ( 7, f1_micro, stdev, 0.000
# ( 7, f1_micro, 95% CI lower, 0.962
# ( 7, f1_micro, 95% CI upper, 0.964
# ( 7, precision, mean, 0.963
# ( 7, precision, stdev, 0.000
# ( 7, precision, 95% CI lower, 0.962
# ( 7, precision, 95% CI upper, 0.964
# ( 7, recall, mean, 0.963
# ( 7, recall, stdev, 0.000
# ( 7, recall, 95% CI lower, 0.962
# ( 7, recall, 95% CI upper, 0.964
# ( 7, brier_score_loss, mean, 0.000
# ( 7, brier_score_loss, stdev, 0.000
# ( 7, brier_score_loss, 95% CI lower, 0.000
# ( 7, brier_score_loss, 95% CI upper, 0.000
# ( 7, log_loss, mean, 0.000
# ( 7, log_loss, stdev, 0.000
# ( 7, log_loss, 95% CI lower, 0.000
# ( 7, log_loss, 95% CI upper, 0.000
# ( 8, roc_auc, mean, 0.992
# ( 8, roc_auc, stdev, 0.000
# ( 8, roc_auc, 95% CI lower, 0.991
# ( 8, roc_auc, 95% CI upper, 0.993
# ( 8, ap, mean, 0.966
# ( 8, ap, stdev, 0.000
# ( 8, ap, 95% CI lower, 0.965
# ( 8, ap, 95% CI upper, 0.967
# ( 8, f1, mean, 0.963
# ( 8, f1, stdev, 0.000
# ( 8, f1, 95% CI lower, 0.962
# ( 8, f1, 95% CI upper, 0.964
# ( 8, f1_macro, mean, 0.963
# ( 8, f1_macro, stdev, 0.000
# ( 8, f1_macro, 95% CI lower, 0.962
# ( 8, f1_macro, 95% CI upper, 0.964
# ( 8, f1_micro, mean, 0.963
# ( 8, f1_micro, stdev, 0.000
# ( 8, f1_micro, 95% CI lower, 0.962
# ( 8, f1_micro, 95% CI upper, 0.964
# ( 8, precision, mean, 0.963
# ( 8, precision, stdev, 0.000
# ( 8, precision, 95% CI lower, 0.962
# ( 8, precision, 95% CI upper, 0.964
# ( 8, recall, mean, 0.963
# ( 8, recall, stdev, 0.000
# ( 8, recall, 95% CI lower, 0.962
# ( 8, recall, 95% CI upper, 0.964
# ( 8, brier_score_loss, mean, 0.000
# ( 8, brier_score_loss, stdev, 0.000
# ( 8, brier_score_loss, 95% CI lower, 0.000
# ( 8, brier_score_loss, 95% CI upper, 0.000
# ( 8, log_loss, mean, 0.000
# ( 8, log_loss, stdev, 0.000
# ( 8, log_loss, 95% CI lower, 0.000
# ( 8, log_loss, 95% CI upper, 0.000
```

Call `run_all()` multiple times in parallel in R

You can use functions from the `future.apply` package to call `run_all()` multiple times in parallel with different parameters. Note that `future.apply` is not compatible on Windows if you haven't already. Then, call `run_all()` with multiple seeds using `future.lapply`:

```
# NOTE: you need seeds for reproducibility
results_multi <- future.apply::future.lapply(FUN= "run_all", function(seed) {
  cv_data_dmp_prepro, "gridsearch", "pipeline", n_jobs=1, seed=seed
  # Fitting 5 folds for each of 1 candidates,
  # Fitting 5 folds for each of 1 candidates,
  # Fitting 5 folds for each of 1 candidates,
  # Fitting 5 folds for each of 1 candidates,
  # Fitting 5 folds for each of 1 candidates,
  # [Parallel(n_jobs=1)]: Using backend SequentialBackend with 1 concurrent jobs.
  # [Parallel(n_jobs=1)]: Done 1 out of 1 | elapsed: 1.1s remaining: 0.0s
  # [Parallel(n_jobs=1)]: Done 5 out of 5 | elapsed: 5.5s remaining: 0.0s
  # [Parallel(n_jobs=1)]: Done 5 out of 5 | elapsed: 5.5s finished
```